Architecture, Block Diagrams and Implementation Guide

8-bit RISC Processor Core with a 5-Stage Pipeline — complete theory, no code

Document 00 of 10 · Read this first · Project files: risc8_5stage_vivado.zip

How to use this document. This document explains **what** the processor is, **how** it is organised and **why** each part exists, using pictures and plain language. It contains no Verilog. Documents 01–09 then explain each source file line by line. After reading this document you should be able to draw the processor on paper and explain every block in a viva.

Contents

1. Basic concepts you need
2. Specification of the processor
3. Instruction set architecture (ISA)
4. Top-level block diagram and file structure
5. Pipelining theory
6. The complete datapath, stage by stage
7. The control unit and control signals
8. Hazards and how they are solved
9. Instruction walk-throughs
10. Design decisions and trade-offs
11. How to build this processor from scratch (method)
12. Implementing it in Vivado, step by step
13. Verification and results
14. Glossary
15. Map of the code documents

1. Basic concepts you need

1.1 What a processor does

A processor (CPU) runs a **program**: a list of instructions stored in memory. It repeats the same cycle forever: **fetch** the next instruction, **decode** it to find out what it means, **execute** it (for example add two numbers), **access memory** if the instruction loads or stores data, and **write back** the result into a register. A counter called the **program counter (PC)** remembers the address of the next instruction.

1.2 Instruction set architecture (ISA) versus microarchitecture

The **ISA** is the contract between software and hardware: which instructions exist, how they are encoded as bits, how many registers there are. The **microarchitecture** is how one particular chip carries out that contract: how many pipeline stages, which shortcuts, which memories. The same ISA can be built as a 1-stage, 3-stage or 5-stage machine. This project uses a custom 8-bit RISC ISA and a classic 5-stage microarchitecture.

1.3 RISC versus CISC

Feature	RISC (this project)	CISC
Instruction length	Fixed (16 bits here)	Variable
Number of instructions	Few and simple (16 here)	Many and complex
Memory access	Only LOAD and STORE touch memory	Most instructions may touch memory
Control unit	Hard-wired logic	Often microprogrammed
Pipelining	Easy, because all instructions look alike	Difficult
Examples	ARM, RISC-V, MIPS, AVR	x86, 8085, 8051

1.4 Harvard versus von Neumann

A **von Neumann** machine keeps instructions and data in one memory, so fetching an instruction and loading data compete for the same port. A **Harvard** machine has two separate memories. This processor is Harvard: the instruction memory (256 × 16 bits) is read every cycle by the fetch stage, while the data memory (256 × 8 bits) is used by the memory stage at the same time.

1.5 Combinational and sequential logic

Combinational logic (gates, adders, multiplexers) produces outputs that depend only on the present inputs, after a small delay. The ALU, the decoder and the hazard unit are combinational. **Sequential** logic (flip-flops, registers) remembers a value and changes only on the rising edge of the clock. The PC, the registers R0–R7 and all pipeline registers are sequential. A digital design is always *registers with combinational logic between them*; describing it that way is called **Register Transfer Level (RTL)** design.

1.6 Clock, reset and the critical path

Every register updates on the rising clock edge. Between two edges, signals must travel through the combinational logic from one register to the next. The slowest such path, the **critical path**, sets the shortest possible clock period. Pipelining shortens that path by cutting the work into stages. **Reset** puts every register in a known starting state (PC = 0, pipeline empty).

2. Specification of the processor

Parameter	Value	Meaning
Data width	8 bits	Registers, ALU and data memory work on numbers 0–255
Instruction width	16 bits, fixed	Every instruction is exactly one 16-bit word
Registers	8 × 8-bit (R0–R7)	Fast storage inside the CPU; R0 is an ordinary register
Program counter	8 bits	Can address 256 instructions
Instruction memory	256 × 16-bit ROM	Holds the program
Data memory	256 × 8-bit RAM	Read by LD, written by ST
Architecture	Harvard	Separate instruction and data memories
Pipeline	5 stages: IF, ID, EX, MEM, WB	Up to 5 instructions in progress at once
Flags	Z (zero), C (carry / borrow)	Set by arithmetic and logic instructions
Instructions	16 (4-bit opcode)	Arithmetic, logic, shift, load, store, jump, branch, halt
Hazard handling	Forwarding, bypass, 1-cycle load-use stall, 2-cycle branch flush	Hardware keeps results correct
Target	Xilinx FPGA through Vivado	Fully synthesizable Verilog-2001

3. Instruction set architecture (ISA)

3.1 Instruction formats

Every instruction is 16 bits. The top 4 bits are always the **opcode**. The remaining bits are interpreted in one of four ways, called formats. Keeping the opcode and the register fields in the same positions in every format lets the decoder read them before it even knows the instruction.

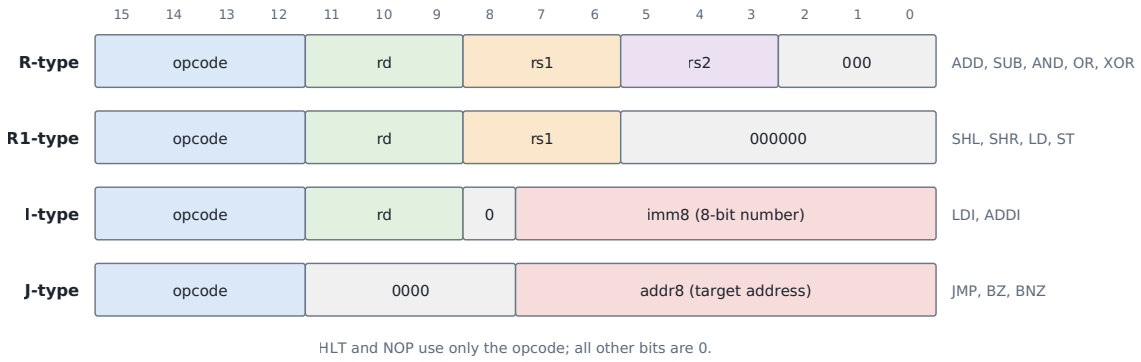


Figure 1. The four instruction formats. Bit 15 is on the left.

Field	Bits	Size	Meaning
opcode	15–12	4	Which of the 16 instructions
rd	11–9	3	Destination register (where the result goes); for ST it is the register to store
rs1	8–6	3	First source register; for LD/ST it holds the memory address
rs2	5–3	3	Second source register
imm8 / addr8	7–0	8	A number inside the instruction (LDI, ADDI) or a jump target (JMP, BZ, BNZ)

3.2 The 16 instructions

Opcode	Instruction	Operation	Flags	Format
0	NOP	Do nothing	–	–
1	ADD rd, rs1, rs2	rd = rs1 + rs2	Z, C	R
2	SUB rd, rs1, rs2	rd = rs1 − rs2	Z, C (borrow)	R
3	AND rd, rs1, rs2	rd = rs1 AND rs2 (bit by bit)	Z	R
4	OR rd, rs1, rs2	rd = rs1 OR rs2	Z	R
5	XOR rd, rs1, rs2	rd = rs1 XOR rs2	Z	R
6	SHL rd, rs1	rd = rs1 shifted left by 1 (× 2)	Z, C = old bit 7	R1
7	SHR rd, rs1	rd = rs1 shifted right by 1 (÷ 2)	Z, C = old bit 0	R1
8	LDI rd, imm8	rd = imm8	–	I
9	ADDI rd, imm8	rd = rd + imm8	Z, C	I
A	LD rd, [rs1]	rd = memory[rs1]	–	R1
B	ST rd, [rs1]	memory[rs1] = rd	–	R1
C	JMP addr8	PC = addr8	–	J
D	BZ addr8	if Z = 1 then PC = addr8	–	J
E	BNZ addr8	if Z = 0 then PC = addr8	–	J
F	HLT	Stop the processor	–	–

3.3 Flags

Z (zero) becomes 1 when the result of an arithmetic, logic or shift instruction is 0. **C (carry)** becomes 1 when an addition overflows 8 bits (for example 255 + 1), when a subtraction needs to borrow (a < b), or holds the bit pushed out by a shift. LDI, LD, ST and the jumps leave the flags unchanged. BZ and BNZ look at Z, which is how loops and if-statements are built.

3.4 Addressing modes

Mode	Example	Where the operand comes from
Register	ADD R3, R1, R2	Registers named in the instruction
Immediate	LDI R1, 5	The number written inside the instruction
Register indirect	LD R5, [R4]	Memory, at the address held in a register
Absolute (branch)	JMP 0x17	The target address is written inside the instruction

3.5 Encoding example

ADD R2, R2, R1: opcode ADD = 0001, rd = R2 = 010, rs1 = R2 = 010, rs2 = R1 = 001, last bits 000. Joined: 0001 010 010 001 000 = 0001 0100 1000 1000 = **0x1488**. **LDI R1, 0x3C**: opcode 1000, rd 001, bit 8 = 0, imm8 = 0011 1100 → **0x823C**.

4. Top-level block diagram and file structure

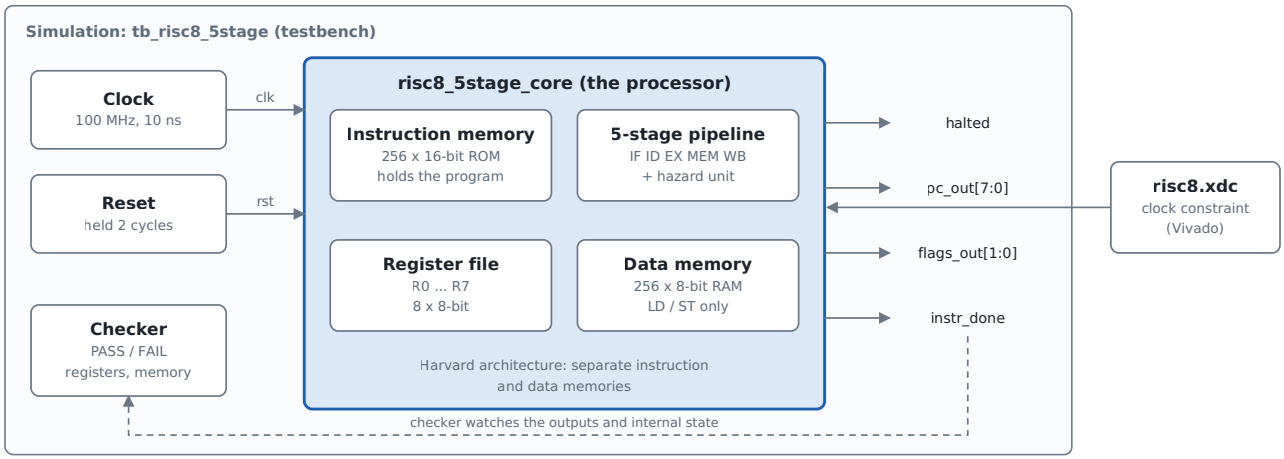


Figure 2. System view: the processor core inside its testbench, with the Vivado constraint file.

The core has only two inputs, **clk** and **rst**. Everything it needs, including the program, is inside it. Its four outputs are for observing it: **halted** (the program finished), **pc_out**, **flags_out** and **instr_done** (pulses each time an instruction completes). On an FPGA board these can drive LEDs.

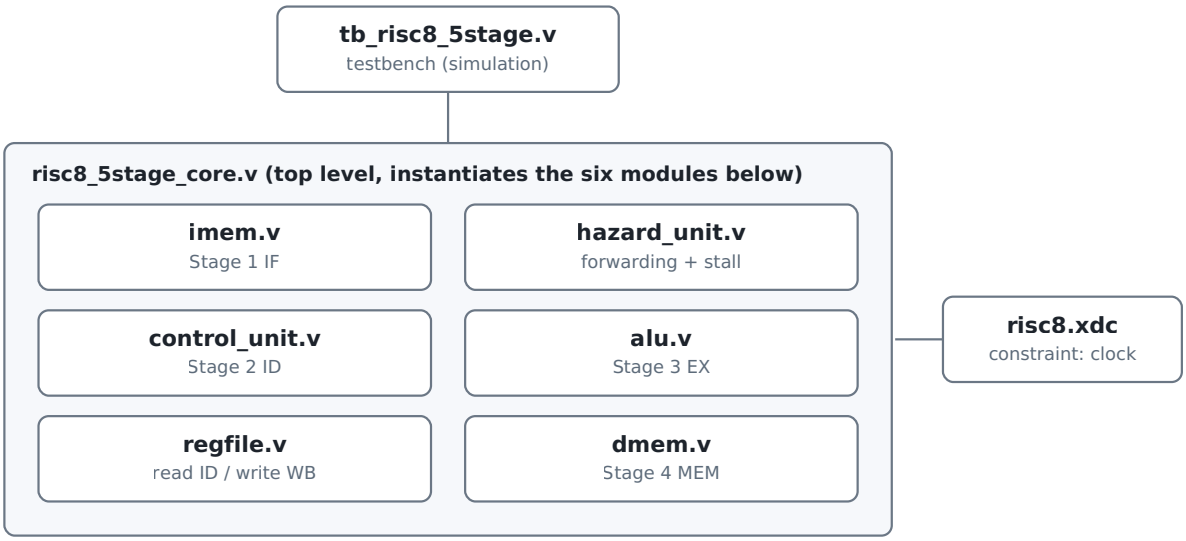


Figure 3. File hierarchy: the testbench creates the core, and the core creates six modules.

File	Module	Kind	Job in one sentence
risc8_5stage_core.v	risc8_5stage_core	Sequential + wiring	Top level: PC, four pipeline registers, flags, branch and halt logic, connects everything
imem.v	imem	Memory (ROM)	Stores the program and returns the instruction at the PC
control_unit.v	control_unit	Combinational	Turns the opcode into control signals
regfile.v	regfile	Sequential	Eight 8-bit registers with two read ports and one write port
hazard_unit.v	hazard_unit	Combinational	Decides forwarding and load-use stalls
alu.v	alu	Combinational	Does the arithmetic and logic; produces Z and C
dmem.v	dmem	Memory (RAM)	256 bytes of data for LD and ST
tb_risc8_5stage.v	tb_risc8_5stage	Testbench	Clock, reset, monitoring, automatic PASS/FAIL
risc8.xdc	–	Constraint	Tells Vivado the target clock (100 MHz)

5. Pipelining theory

5.1 The idea

Without a pipeline, one instruction would go through all five jobs before the next one starts. With a pipeline, the five jobs are done by five separate pieces of hardware (stages), each working on a different instruction, like an assembly line. Between the stages are **pipeline registers** that hold an instruction's information until the next clock edge pushes it one stage forward.

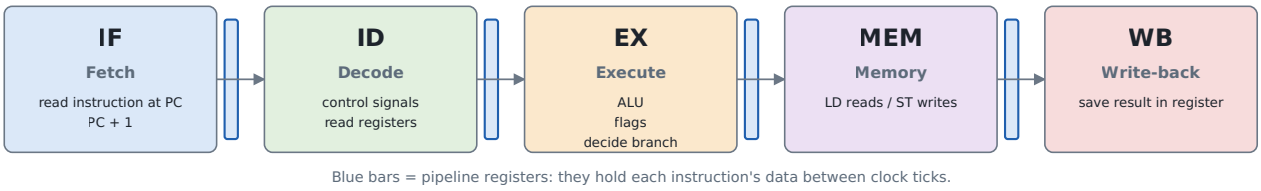


Figure 4. The five stages and what each one does.

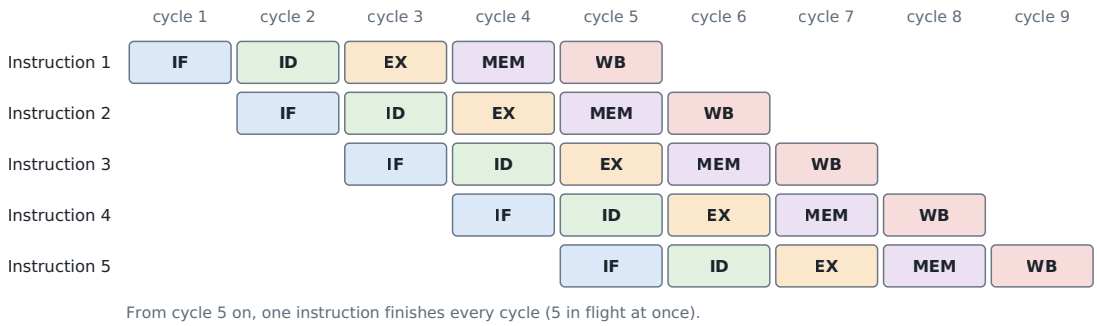


Figure 5. Pipeline timing: five instructions overlap.

5.2 Throughput, latency and CPI

Latency (time for one instruction) is still 5 cycles. **Throughput** becomes one instruction per cycle once the pipeline is full. For n instructions on a k-stage pipeline the ideal time is k + (n − 1) cycles instead of n × k, so the speedup approaches k. **CPI** (cycles per instruction) is ideally 1; hazards add

extra cycles. In our test program CPI = 1.50: 34 instructions take 51 cycles (4 fill cycles, 12 flush cycles, 1 stall cycle).

5.3 Why exactly these five stages

Each stage contains one major piece of hardware, so their delays are balanced: memory read (IF), decoder and register read (ID), ALU (EX), memory again (MEM) and register write (WB). Putting the ALU and the data memory into separate stages is what makes the 5-stage design faster than a 3-stage one; on an iCE40 FPGA the clock rose from 48.8 MHz to 73.2 MHz.

5.4 Valid bits and bubbles

Each pipeline register carries a **valid** bit. If valid = 0 the slot is empty: a **bubble**. A bubble moves through the stages like an instruction but every write-enable it carries is forced to 0, so it cannot change a register, memory or the flags. The hardware creates bubbles on purpose to stall or to cancel (flush) instructions.

6. The complete datapath

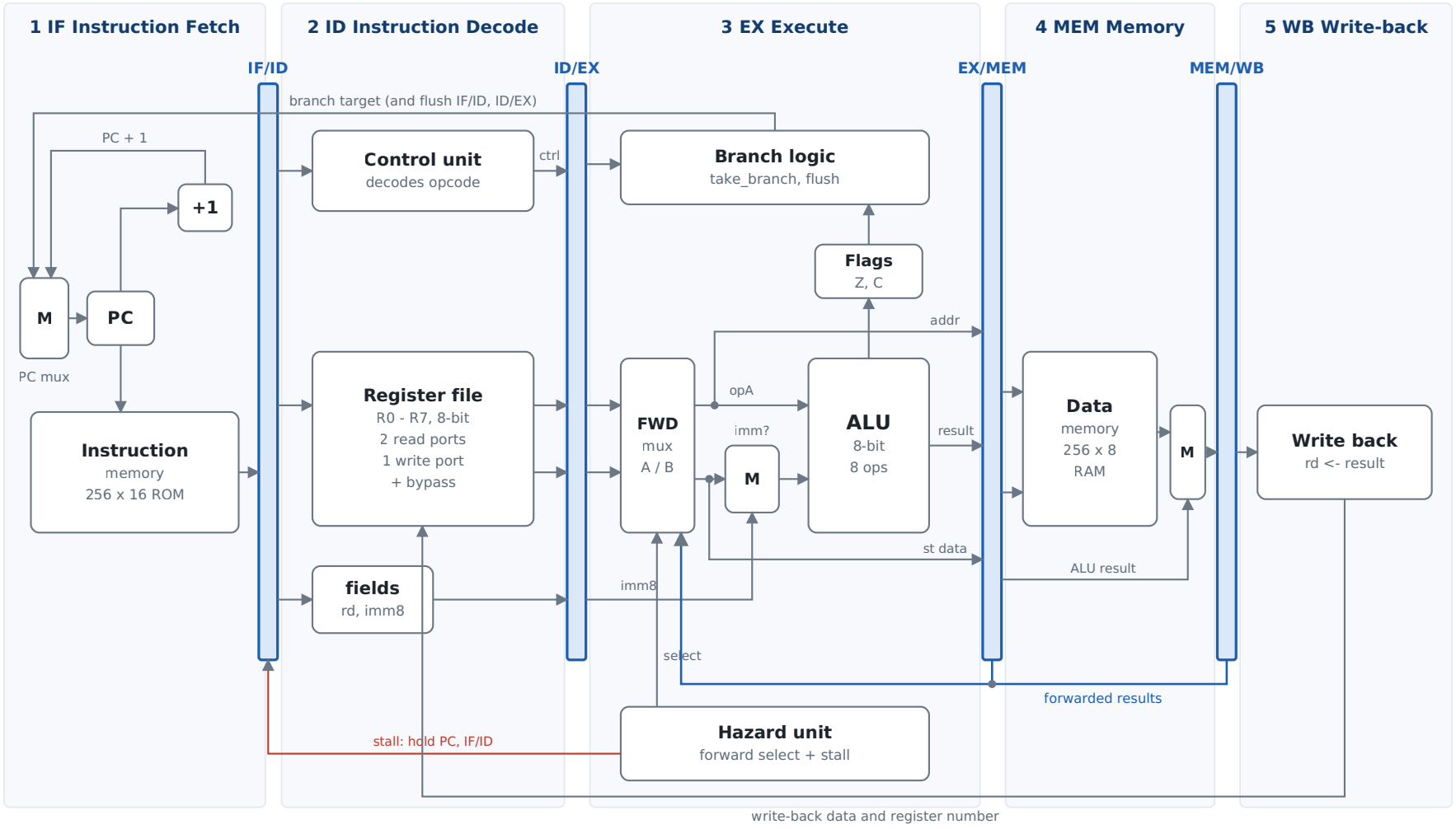


Figure 6. Complete 5-stage datapath. Blue bars are the four pipeline registers. M = multiplexer. Red line = stall. Blue bus = forwarded results.

6.1 Stage 1 – Instruction Fetch (IF)

The PC sends its address to the instruction memory, which returns the 16-bit instruction in the same cycle. At the clock edge the instruction is stored in the **IF/ID** register with valid = 1, and the PC is updated. The **PC multiplexer** chooses the next PC:

Condition (in priority order)	Next PC	IF/ID gets
A branch or jump is taken in EX	branch target	bubble (wrong-path instruction cancelled)
HLT has reached EX	PC unchanged	bubble (nothing more is fetched)
Load-use stall	PC unchanged	unchanged (the same instruction waits)
Normal	PC + 1	the fetched instruction

6.2 Stage 2 – Instruction Decode (ID)

The instruction in IF/ID is split into its fields. The **control unit** turns the opcode into control signals. The **register file** is read on two ports. Two special cases choose which register each port reads: ADDI reads *rd* on port A (because it computes rd = rd + imm8), and ST reads *rd* on port B (the value to store). The register file also has a **bypass**: if the WB stage is writing the very register being read, the new value is passed straight through. Everything is stored in the **ID/EX** register.

6.3 Stage 3 – Execute (EX)

Two **forwarding multiplexers** pick each operand from one of three places: the value read in ID, the newer result waiting in EX/MEM, or the result in MEM/WB. A second multiplexer chooses whether the ALU's B input is the register value or the immediate number. The **ALU** computes the result and the Z and C flags; the flags are stored at the end of the cycle if the instruction is allowed to change them. The **branch logic** decides whether a JMP, BZ or BNZ is taken. Results, the memory address and the store data go into **EX/MEM**.

6.4 Stage 4 – Memory access (MEM)

For LD the data memory is read at the address in EX/MEM; for ST it is written at the clock edge. A multiplexer passes either the loaded byte (LD) or the ALU result (all others) into **MEM/WB**.

6.5 Stage 5 – Write-back (WB)

If the instruction writes a register, MEM/WB drives the register file's write port and the value is stored at the next clock edge. The same value is also sent back to the forwarding multiplexers and to the register-file bypass.

6.6 Contents of the pipeline registers

Register	Contents	Width
IF/ID	instruction (16), valid (1)	17 bits
ID/EX	valid, 14 control bits, rd, two register numbers, two register values, imm8	48 bits
EX/MEM	valid, reg_we, mem_to_reg, mem_we, halt, rd, ALU result, address, store data	32 bits
MEM/WB	valid, reg_we, halt, rd, result	14 bits

7. The control unit and control signals

The control unit is a lookup table from opcode to control signals. It starts with every signal off (which is a NOP) and switches on only what each instruction needs. Because it has no memory, it is pure combinational logic: a **hard-wired** control unit, typical of RISC.

Signal	Meaning when 1
alu_op (3 bits)	Which ALU operation: 0 ADD, 1 SUB, 2 AND, 3 OR, 4 XOR, 5 SHL, 6 SHR, 7 pass B
alu_src_imm	ALU input B is the immediate number, not a register
ra_is_rd / rb_is_rd	Read port A / B reads register rd instead of rs1 / rs2
uses_a / uses_b	The instruction really reads port A / B (used by the hazard unit)
reg_we	Write the result into register rd in WB
mem_to_reg	The result comes from data memory (LD)
mem_we	Write data memory in MEM (ST)
flag_we	Update Z and C in EX
jump / br_z / br_nz	Instruction is JMP / BZ / BNZ
halt	Instruction is HLT

7.1 Control signal table

Instr.	alu op	alu_src _imm	ra_is _rd	rb_is _rd	uses _a	uses _b	reg _we	mem_to _reg	mem _we	flag _we	branch / halt
NOP	–										
ADD	0				1	1	1			1	
SUB	1				1	1	1			1	
AND	2				1	1	1			1	
OR	3				1	1	1			1	
XOR	4				1	1	1			1	
SHL	5				1		1			1	
SHR	6				1		1			1	
LDI	7	1					1				
ADDI	0	1	1		1		1			1	
LD	–				1		1	1			
ST	–			1	1	1			1		
JMP	–										jump
BZ	–										br_z
BNZ	–										br_nz
HLT	–										halt

A blank cell means 0. alu_op – means the ALU result is not used.

7.2 Control signals travel with the instruction

The decoder works in ID, but many signals are needed later. So they are copied into the pipeline registers and travel along with their instruction. Each stage uses its own group and passes the rest on. A bubble has all its write-enables forced to 0.

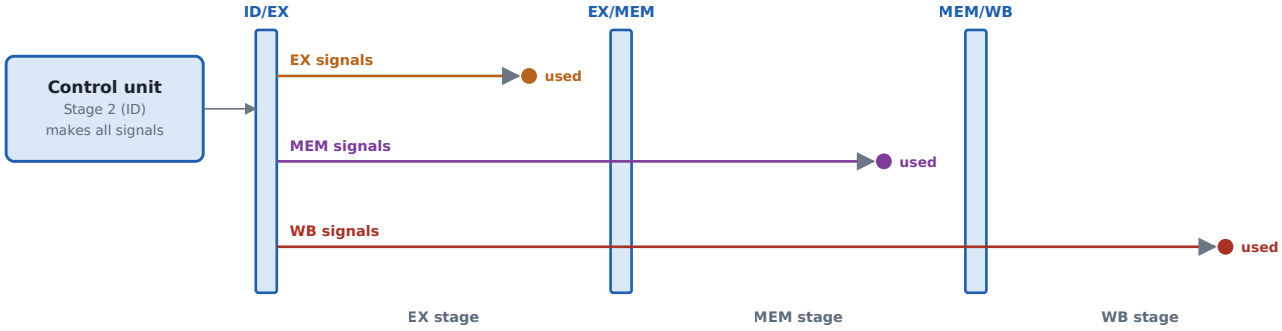


Figure 7. EX signals are used one stage later, MEM signals two stages later, WB signals three stages later.

Group	Signals	Used in
EX	alu_op, alu_src_imm, flag_we, jump, br_z, br_nz, halt	Stage 3
MEM	mem_we, mem_to_reg	Stage 4
WB	reg_we, rd	Stage 5

8. Hazards and how they are solved

A **hazard** is a situation in which the overlapped instructions would produce a wrong result if the hardware did nothing. There are three kinds.

Kind	Cause	Solution here	Cost
Structural	Two stages need the same resource	Harvard memories; register file with separate read and write ports	0
Data (read after write)	An instruction needs a result that is not written back yet	Forwarding from EX/MEM and MEM/WB; register-file bypass	0 cycles
Data, load-use	The value comes from memory and is needed immediately	1-cycle stall, then forwarding	1 cycle
Control	A taken branch makes the already-fetched instructions wrong	Flush the two younger instructions	2 cycles

8.1 Forwarding

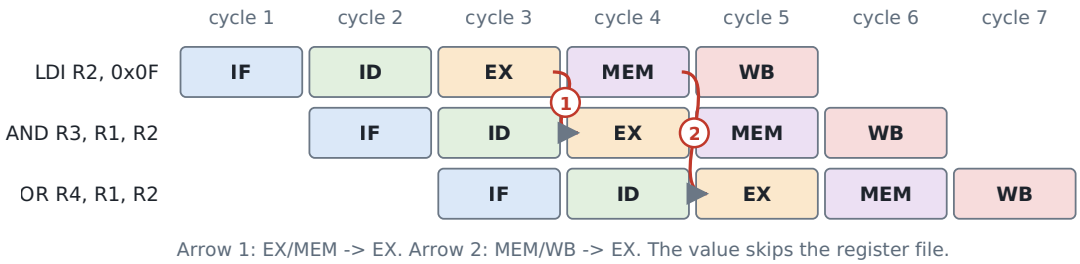


Figure 8. Forwarding: AND and OR receive R2 before LDI writes it.

The hazard unit compares the source register numbers of the instruction in EX with the destination register numbers of the instructions in MEM and WB. The rules, checked separately for operand A and operand B, are:

1. If the instruction in **MEM** will write a register, is not a load, and its register equals the one we read, take its ALU result from EX/MEM (newest value wins).
2. Otherwise, if the instruction in **WB** will write that register, take the MEM/WB result.
3. Otherwise use the value read in ID.

Forwarding is done only for operands the instruction really reads (uses_a / uses_b), so unused register fields never cause a false match.

8.2 Register-file bypass

If an instruction three places behind reads a register in ID in the same cycle as WB writes it, the register file itself returns the new value. Together with the two forwarding paths, every distance between producer and consumer (1, 2 or 3 instructions) is covered without waiting.

8.3 Load-use stall

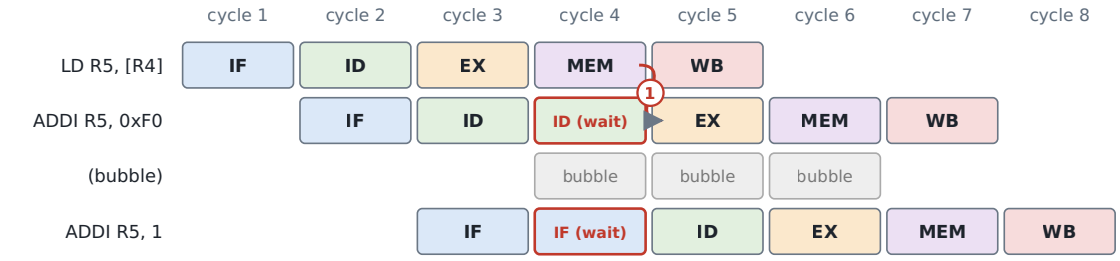


Figure 9. Load-use stall: one bubble is inserted, then the value is forwarded.

A load gets its data only at the end of MEM. If the very next instruction needs it in EX, the data does not exist yet. The hazard unit detects: *a load is in EX, and the instruction in ID reads the register the load will write*. It then raises **stall** for one cycle: the PC and IF/ID keep their values, and a bubble enters ID/EX. One cycle later the load is in WB and normal forwarding supplies the value.

8.4 Control hazards: branch flush

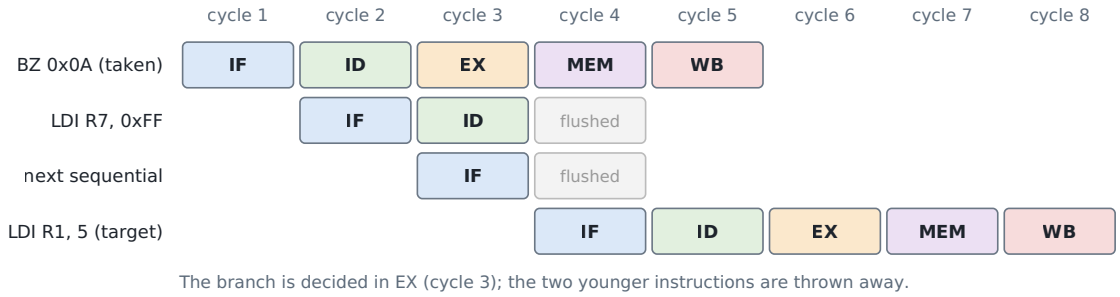


Figure 10. A taken branch flushes the two instructions fetched after it.

The processor keeps fetching sequentially while a branch moves down the pipeline. The branch is decided in EX, where the flags are up to date. If it is taken, the PC is loaded with the target and the two younger instructions (in IF/ID and ID/EX) become bubbles. They never reach MEM or WB, so they cannot change memory or registers. Cost: 2 cycles per taken branch; not-taken branches cost nothing.

8.5 Halting

When HLT reaches EX, fetching stops and younger instructions are squashed, but the older ones in MEM and WB still finish. When HLT itself reaches WB, the **halted** output becomes 1 and the whole pipeline freezes. This guarantees that every instruction before HLT completes.

9. Instruction walk-throughs

Stage	ADD R3, R1, R2	LD R5, [R4]	ST R2, [R4]	BZ 0x0A
IF	fetch 0x3650	fetch 0xAB00	fetch 0xB500	fetch 0xD00A
ID	read R1, R2; reg_we=1, flag_we=1, alu_op=ADD	read R4; reg_we=1, mem_to_reg=1	read R4 (A) and R2 (B); mem_we=1	br_z=1; target 0x0A
EX	ALU: R1 + R2; set Z, C	address = R4 passed on	address = R4, data = R2 passed on	if Z = 1: PC = 0x0A, flush 2
MEM	pass result	read memory[R4]	write memory[R4] = R2	–
WB	R3 = result	R5 = loaded byte	nothing	nothing

10. Design decisions and trade-offs

Decision	Why	Alternative
Branches resolved in EX	The Z flag is produced by the ALU in EX, so the decision is always based on up-to-date flags	Decide in ID with extra compare logic: 1-cycle penalty, more hardware
Fixed 16-bit instructions	8 bits are too few for opcode plus registers; fixed length keeps decoding simple	Variable length (CISC): harder to pipeline
Asynchronous memory read	Instruction and data are available in the same cycle as the address	Synchronous block RAM: one more cycle of latency
Write-through register bypass	Removes the distance-3 hazard with a tiny multiplexer	Write in first half of cycle, read in second half
Forwarding gated by uses_a / uses_b	No false forwards or stalls from unused fields	Simpler logic, but occasional needless stalls
Program built into imem.v	No external file; Vivado cannot lose it	$readmemh from a .hex file
Debug outputs (pc_out, flags_out, instr_done)	Observable on LEDs; no unused-signal warnings	Internal signals only

11. How to build this processor from scratch (method)

Follow these steps in order. The source files were written in exactly this order, from the smallest independent block to the top level.

Step	What to do
1. Write the specification	Decide data width, number of registers, memory sizes and which instructions you need (Section 2).
2. Design the ISA	Choose the instruction formats and assign opcodes; write the encoding table (Section 3).
3. Draw a single-cycle datapath	On paper, connect PC, instruction memory, decoder, register file, ALU, data memory and write-back so that one instruction completes per (long) cycle.
4. Cut it into five stages	Insert the four pipeline registers at the natural boundaries: after fetch, after decode, after the ALU and after memory (Figure 6).
5. Write the control table	For every instruction list every control signal (Section 7.1) and decide in which stage each one is used.
6. Find the hazards	For each pair of dependent instructions at distance 1, 2 and 3, check where the value is and where it is needed; decide forward, bypass or stall. Decide where branches are resolved.
7. Write the leaf modules	ALU, register file, data memory, instruction memory: each is small and independent. Test each one alone in simulation.
8. Write the control unit	A case statement on the opcode with defaults first.
9. Write the hazard unit	Comparators for forwarding (EX vs MEM and WB) and for load-use (EX load vs ID).
10. Write the top level	Declare the pipeline registers, instantiate all modules, add the multiplexers and the single clocked block that updates every register with the priority flush > halt > stall > normal.
11. Write a test program	Cover every instruction, dependencies at distances 1–3, a load-use pair, taken and not-taken branches, and wrong-path instructions that must not execute. Work out the expected final register values by hand.
12. Write a self-checking testbench	Generate clock and reset, print the pipeline every cycle, wait for halted, compare every register with the expected value and print PASS/FAIL.
13. Simulate and debug	Use the cycle trace and waveforms; check that stalls and flushes happen exactly where expected.
14. Synthesize and implement	Check for latches, utilisation and timing (Section 12).

12. Implementing it in Vivado, step by step

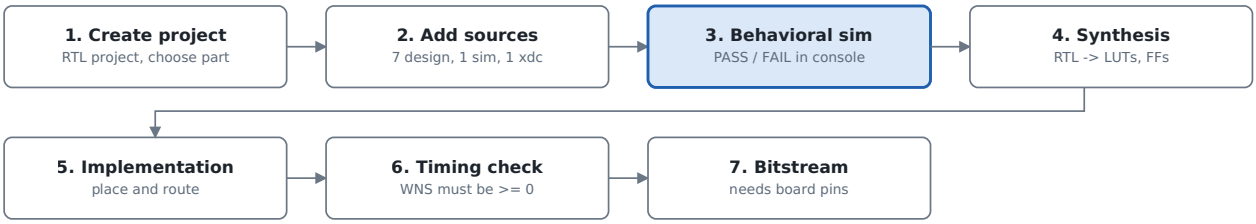


Figure 11. The Vivado design flow used for this project.

12.1 Create the project

1. Start Vivado → **Create Project** → Next → name it (no spaces in the path) → **RTL Project**.
2. **Add Sources**: Add Files → select the 7 files in *design_sources/* → tick **Copy sources into project**.
3. **Add Constraints**: select *constraints/risc8.xdc*.
4. **Default Part**: choose your board's FPGA, or any Artix-7 such as xc7a35tcpg236-1 (Basys-3).
5. Finish. In the Sources window **risc8_5stage_core** should be shown in bold as the top module.
6. **File → Add Sources → Add or create simulation sources** → select *sim_sources/tb_risc8_5stage.v*.

12.2 Behavioral simulation

1. Sources → Simulation Sources → right-click **tb_risc8_5stage** → **Set as Top**.
2. Flow Navigator → **Run Simulation → Run Behavioral Simulation**.
3. The test stops by itself at 545 ns. The Tcl Console shows one line per cycle and finally *** **ALL TESTS PASSED** ***.
4. For waveforms: in the Scope panel open tb_risc8_5stage → dut, select signals such as pc, stall, take_branch, fwd_a_sel, ifid_valid, add them to the wave window, then Restart and Run All.

12.3 Synthesis, implementation and timing

1. **Run Synthesis**. Open **Report Utilization**: LUTs, flip-flops and LUT-RAM (the data memory).
2. Check the messages: there must be no 'latch' warnings. Notes about unused or constant signals are normal.
3. **Run Implementation**, then **Report Timing Summary**. The worst negative slack (WNS) should be positive for the 100 MHz target.
4. **Generate Bitstream** requires pin assignments for your board in risc8.xdc (clk on the board oscillator, rst on a button, outputs on LEDs).

Tip for hardware demos. On a real board the program finishes in about half a microsecond. To watch it on LEDs, add a clock divider (for example to 1 Hz) between the board clock and the core's clk input.

13. Verification and results

The test program covers every instruction class, dependencies at distances 1, 2 and 3, a load-use pair, a loop that sums 5+4+3+2+1, taken branches with wrong-path instructions that must be cancelled, a carry, a jump and HLT. The testbench checks 9 results automatically.

Check	Expected	Result
R1, R2, R3, R4	0x00, 0x0F, 0x01, 0x20	PASS
R5, R6, R7	0x00, 0x66, 0x00	PASS
Memory[0x20]	0x0F	PASS
Flags Z, C	1, 1	PASS

Metric	Value
Cycles / instructions	51 / 34
CPI	1.50
Load-use stalls	1
Taken branches (2-cycle flush each)	6
Forwarded operands	12
Random programs compared with a reference model	500 of 500 identical
Lint (Verilator -Wall), compile (Icarus -Wall), synthesis (Yosys, Xilinx 7-series)	0 errors, 0 warnings

14. Glossary

Term	Meaning
ALU	Arithmetic Logic Unit: the circuit that adds, subtracts and does logic
Bubble	An empty pipeline slot (valid = 0) that does nothing
Bypass	Passing a value being written straight to a reader in the same cycle
CPI	Cycles per instruction
Critical path	Slowest register-to-register path; limits clock speed
Flush	Cancelling wrongly fetched instructions by turning them into bubbles
Forwarding	Sending a result from a later pipeline register back to the EX stage
Hazard	A pipeline situation that would give a wrong result if not handled
Immediate	A number stored inside the instruction itself
Latch (unwanted)	Memory created by mistake when combinational logic does not assign an output in every case
Opcode	The part of the instruction that says which operation to do
Pipeline register	Register between two stages that holds an instruction's data
RTL	Register Transfer Level: describing hardware as registers and the logic between them
Stall	Holding an instruction in its stage for an extra cycle
Synthesis	Turning RTL code into gates and flip-flops
Testbench	Simulation-only code that drives and checks the design
WNS	Worst negative slack: timing margin after implementation; must be ≥ 0

15. Map of the code documents

Recommended reading order for implementing: 01 → 02 → 03 → 04 → 05 → 06 → 07 → 08 → 09. This follows the dependency order: each file only uses files explained before it.

Document	File explained	Read it to learn
01	alu.v	How arithmetic, logic, shifts and flags are computed
02	regfile.v	How registers are stored, read, written and bypassed
03	imem.v	How the program is stored and how to change it
04	dmem.v	How load and store access memory
05	control_unit.v	How the opcode becomes control signals
06	hazard_unit.v	How forwarding and stalls are decided
07	risc8_5stage_core.v	How everything is connected and how the pipeline moves
08	tb_risc8_5stage.v	How the design is tested and checked automatically
09	risc8.xdc	How Vivado is told the clock speed and pins